

Week 3 Evaluation Report

ID2221 Data-Intensive Computing | September 2026

1. Experimental method

We evaluated update correctness, refresh cost and the overhead of validation and monitoring on the course datasets. The local environment used Python 3.12.14, Java 17.0.20.1, PySpark 4.0.4 and Delta Lake 4.0.1 on Windows, with local[4] and temporary storage on D:. Each of 2 trials started three workspaces from the same validated baseline and applied the same four update files.

The three variants were incremental refresh, full Gold/product rebuild, and incremental refresh with event monitoring disabled. They share the same ingestion, orchestration and publication code. In the first trial their order was incremental, full, monitoring-off; the second reversed it. Spark caches were cleared between variants, but operating-system caches were not. This controls the work performed and reduces order bias without eliminating local timing noise. The monitoring comparison also checks that disabling logging leaves analytical results unchanged.

2. Validation and update outcomes

The baseline revalidates the historical inputs using the Week 3 rules. Duplicates and other rejected records are listed separately below.

Dataset	Input rows	Accepted	Duplicates	Other rejects
Taxi zones	265	263	0	2
Weather	8,784	8,784	0	0
Air quality	8,139,551	7,928,224	0	211,327
Taxi trips	9,554,778	9,323,689	1	231,088

Invalid historical rows are quarantined and excluded from analysis. A row may violate multiple rules, so failure counts can exceed rejected-row counts. The controlled release gave the following outcomes in the first trial; both trials use the same update files and record outcomes.

Dataset	Processed	Inserted	Updated	Duplicates	Rejected
Taxi zones	3	0	3	0	0
Weather	174	168	6	0	0
Air quality	507	504	3	0	0
Taxi trips	859,930	716,608	0	143,322	0

Taxi updates add 7.5% new trips and 1.5% exact copies of the original rows. Weather adds humidity, air quality adds AQI, and historical corrections exercise propagation to existing trips. Invalid examples remain in separate tests.

3. Update and analytical refresh performance

Trial	Incremental total (s)	Full total (s)	Incremental Gold (s)	Full Gold (s)
1	1480.4	247.2	244.4	49.9
2	514.5	277.9	253.2	50.0

In trial 2, incremental processing took 514.5 s, compared with 277.9 s for the full workflow. The incremental path touched 2,528,301 trips: zone-label corrections can spread to a large part of Gold. These results support measuring correction size before choosing MERGE or a full rebuild; we have not established a switching threshold.

Product	Refresh mode	Trial 1 (s)	Trial 2 (s)
Daily mobility	groups	24.3	20.8
Taxi-zone statistics	groups	34.7	31.3
Weather impact	full	964.2	16.7
Air-quality impact	groups	13.4	14.1

Group refresh replaces affected aggregation groups. Weather is rebuilt when the analysis window advances because its observed-hour denominators change. Full product rebuilding took a median of 68.8 s. Product timings include registry updates in both variants.

The first trial coincided with a prolonged delay in the host terminal's response during weather refresh. We retain the observation rather than remove it after seeing the result. Together with the small sample, this limits attribution of timing differences to the refresh strategy alone.

4. Validation and monitoring overhead

For validation, we fully materialised the same transformed input and reference rows before timing either arm. Both arms consume every column through the same row-hash aggregation. The enabled arm checks schema and values, then recombines accepted and rejected rows so both sinks have the same input multiset. Preparation is measured separately and excluded. This estimates validation computation over prepared inputs; it excludes transformation, MERGE and quarantine writes.

Trial	Validation on (s)	Validation off (s)	Difference (s)	Monitoring-off total (s)	Monitoring difference (s)
1	25.17	1.07	24.11	512.8	967.6
2	25.12	0.92	24.20	531.9	-17.4

The median additional validation time was 24.15 s. The observed monitoring difference is the incremental total in Section 3 minus the otherwise identical monitoring-off run above. Direct event writes took 21.1 s at the median. Publication and product registry writes remain enabled in both arms. All raw differences, including any negative values, are retained; two trials are insufficient to separate small overheads from scheduling, disk and cache variation with confidence. The first monitoring difference also includes the host delay noted in Section 3, so it cannot be attributed entirely to event logging. The negative difference in trial 2 does not mean logging improves performance; it shows that paired wall times do not give a precise estimate of this cost on the local host.

5. Storage and monitoring results

We counted every file under the trial data directory and reconciled the categories with its total. The table shows trial 2's change from the same baseline. No VACUUM ran: obsolete data, CDF, logs, quarantine and staging are retained. Sizes are logical file bytes, with hard links counted per path, not additional disk allocation. Input files and evaluation clones are outside this platform total.

Component	Change (bytes)
Active business data	135,804,335
Retained business data files	853,615,455
Business CDF files	273,068,020
Business Delta logs and other files	7,867,427
Quarantine	13,530,912
Staging and change keys	125,228,538
Monitoring, publication and registry	491,584
Schema manifests and other metadata	0
Total	1,409,606,271

Active business data grew by 135,804,335 bytes. Operational and retained files grew by 1,273,801,936 bytes. Separating these quantities avoids treating all new data as infrastructure overhead. Storage cost depends on retention; the inventory captures this experiment before cleanup.

The saved production monitoring snapshot reports validation events in these execution counts: Taxi trips: 3/3, Air quality: 3/3, Taxi zones: 3/4, Weather: 1/4. Taxi trips had the longest Bronze duration, 389.5 s. These values mix baseline loads and retries. Duplicate replays count as validation events even with zero invalid rows, so we interpret them alongside rejection reasons, stage duration and status. Source freshness, shuffle spill and quarantine age would help diagnose a deployed system, but are not measured here.

6. Correctness and maintainability

Both refresh variants and both monitoring settings produced matching Gold hash summaries, four product results and all six query results. Final Gold contains 10,040,297 rows. Hash summaries are strong evidence of agreement rather than collision-free row-by-row proof; numeric product results are compared with floating-point tolerance.

Recovery tests use real Delta tables and inject failures after Gold and after a product commit. They check that the old publication remains visible, retries match complete product recomputation, pending keys survive a new batch, and a completed replay performs no analytical refresh. A multi-commit reversal test checks that intermediate product groups remain eligible for repair.

Week 1's common schema and configuration-driven loader reduced changes to ingestion and validation. Cross-table publication and correction propagation required the most work. New datasets can reuse existing rule types but still need keys, references and sometimes transforms or integration code. We would next add a rule registry and a measured choice between partial and full refresh. Multiple writers would require coordination; recovery also depends on retained Delta history. This local experiment does not establish cluster scalability. More cities would require city-aware keys, partition pruning, skew tests and file compaction. The weather timezone remains an unverified UTC assumption.